

Java OOP Mastery - Phase 4

ABSTRACTION | Abstract Classes | Interfaces | Default/Static Methods | Contracts | Polymorphism

Goal: Understand abstraction so well that you can explain not only what Java syntax does, but why the design exists and when you would choose each tool.

Four pillars status: Encapsulation - completed | Inheritance - completed | Polymorphism - completed |
Abstraction - this phase

1. What Problem Does Abstraction Solve?

Abstraction is mainly about showing the **essential behavior** while hiding unnecessary implementation details. The caller should know what an object can do, without being forced to understand every internal step required to do it.

```
class Car {  
    void start() {  
        // fuel injection  
        // ignition  
        // sensors  
        // temperature checks  
        // engine internals  
    }  
}
```

The caller only needs:

```
Car car = new Car();  
car.start();
```

The caller does not need to know how the engine internally performs the start operation. The public operation is simple even if the implementation is complicated.

Mental model:

```
Caller  
  |  
  v  
essential operation: start()  
  |  
  v  
hidden implementation details
```

2. Encapsulation vs Abstraction

Encapsulation asks: How do I protect and control internal state? Example: `private double balance;`

Abstraction asks: What does the outside world actually need to know and use? Example: `payment.pay();` without exposing the internal payment process.

Encapsulation
-> protect/control internal state
-> private fields
-> controlled methods

Abstraction
-> expose essential behavior
-> hide unnecessary implementation complexity
-> abstract classes / interfaces

They work together. Encapsulation protects internals; abstraction shapes the useful public model of the object.

3. Java's Two Main Abstraction Tools

Java gives us two major mechanisms for abstraction: **abstract classes** and **interfaces**. They overlap, but they solve somewhat different design problems.

4. Abstract Classes

```
abstract class Animal {  
    abstract void sound();  
  
    void eat() {  
        System.out.println("Eating");  
    }  
}
```

Here `sound()` is abstract: the parent declares the required behavior but does not provide the implementation. `eat()` is a normal concrete method with shared implementation.

Important: an abstract class cannot be directly instantiated.

```
Animal a = new Animal(); // ERROR  
  
Animal a = new Dog();    // OK, if Dog is concrete
```

5. Why Does the Child Have to Implement the Abstract Method?

```
abstract class Animal {  
    abstract void sound();  
}  
  
class Dog extends Animal {  
    @Override  
    void sound() {  
        System.out.println("Bark");  
    }  
}
```

The abstract parent creates a requirement: a concrete child must fulfill the missing behavior. If the child does not implement all required abstract methods, the child must itself be declared abstract.

```
abstract class Animal {  
    abstract void sound();  
}  
  
abstract class Mammal extends Animal {  
    // Mammal may leave sound() abstract  
}
```

6. An Abstract Class Can Contain Normal Things Too

```
abstract class Animal {  
    String name;  
  
    Animal(String name) {  
        this.name = name;  
    }  
  
    void eat() {  
        System.out.println("Eating");  
    }  
  
    abstract void sound();  
}
```

An abstract class can have fields, constructors, concrete methods, abstract methods, static members, and more. 'Abstract class' does not mean 'everything inside must be abstract.'

7. Why Can an Abstract Class Have a Constructor?

```
abstract class Animal {  
  
    Animal() {  
        System.out.println("Animal constructor");  
    }  
  
    abstract void sound();  
}  
  
class Dog extends Animal {  
  
    Dog() {  
        System.out.println("Dog constructor");  
    }  
  
    @Override  
    void sound() {  
        System.out.println("Bark");  
    }  
}  
  
Dog d = new Dog();
```

Output:

```
Animal constructor  
Dog constructor
```

You cannot instantiate `Animal` directly, but the **Animal part of a Dog object still has to be initialized**. Therefore the abstract class constructor participates in the child object's construction sequence.

8. Interface - The Contract Idea

```
interface Payment {  
    void pay();  
}  
  
class UPI implements Payment {  
  
    @Override  
    public void pay() {  
        System.out.println("UPI payment");  
    }  
}  
  
class CardPayment implements Payment {  
  
    @Override  
    public void pay() {  
        System.out.println("Card payment");  
    }  
}
```

The interface says: a type that implements `Payment` must provide the required `pay()` behavior. It does not force the caller to know the internal implementation details of UPI, cards, wallets, or other providers.

```
Payment p1 = new UPI();  
Payment p2 = new CardPayment();  
  
p1.pay();  
p2.pay();
```

This is abstraction working together with polymorphism: the reference uses the common Payment type, while the actual object supplies the concrete behavior.

9. Abstract Class vs Interface - Design Thinking

Abstract class: useful when several types belong to a common class hierarchy and should share state and/or implementation.

```
abstract class Animal {
    String name;

    void eat() {
        System.out.println("Eating");
    }

    abstract void sound();
}
```

Interface: useful when you want to express a contract or capability that multiple, potentially unrelated classes can provide.

```
interface Flyable {
    void fly();
}

class Bird implements Flyable {
    public void fly() {
        System.out.println("Bird flying");
    }
}

class Drone implements Flyable {
    public void fly() {
        System.out.println("Drone flying");
    }
}
```

A useful mental shortcut is: abstract class often describes what something **is** in a class hierarchy; an interface often describes what something **can do**. This is a design heuristic, not an absolute law.

10. extends vs implements

```
class Dog extends Animal {
}
```

`extends` creates a class-to-class inheritance relationship.

```
class UPI implements Payment {
}
```

`implements` means the class agrees to satisfy the interface contract.

```
class Dog extends Animal implements Pet, Trainable {
}
```

A class can extend one class, but it can implement multiple interfaces.

11. Interface Methods Today

Do not memorize the old simplified statement 'interfaces contain only abstract methods.' Modern Java interfaces can contain abstract methods, default methods, static methods, and private helper methods.

```
interface Payment {

    void pay();                                // abstract
}
```

```

    default void receipt() {                // default implementation
        System.out.println("Receipt");
    }

    static void info() {                    // static method
        System.out.println("Payment");
    }

    private void helper() {                // private helper
        System.out.println("Internal helper");
    }
}

```

12. Default Methods

```

interface Payment {

    void pay();

    default void receipt() {
        System.out.println("Receipt");
    }
}

class UPI implements Payment {

    @Override
    public void pay() {
        System.out.println("UPI");
    }
}

UPI upi = new UPI();

upi.pay();
upi.receipt();

```

A default method already has an implementation, so an implementing class does not have to override it. The implementing class can still override it if it needs different behavior.

Why do default methods exist? They allow interfaces to evolve by adding behavior without forcing every existing implementation to immediately provide a new implementation.

13. Static Methods in Interfaces

```

interface Payment {

    static void info() {
        System.out.println("Payment system");
    }
}

Payment.info();

```

The static method belongs to the interface itself. Think `Payment.info()`, not polymorphic instance dispatch through a `Payment` object.

14. Private Methods in Interfaces

```

interface Payment {

    default void receipt() {
        formatReceipt();
    }

    private void formatReceipt() {
        System.out.println("Formatting");
    }
}

```

```
}  
}
```

A private interface method is an internal helper for code inside the interface, commonly used by default methods. It is not part of the public contract exposed to implementing classes or callers.

15. Interface Fields

```
interface Payment {  
    int MAX_AMOUNT = 100000;  
}
```

Interface fields are implicitly public static final. Conceptually, the declaration is equivalent to:

```
public static final int MAX_AMOUNT = 100000;
```

So the field is a constant-like class-level value associated with the interface, not a mutable per-object instance field.

16. Interface Method Visibility

```
interface Payment {  
    void pay();  
}  
  
class UPI implements Payment {  
  
    @Override  
    public void pay() {  
        System.out.println("UPI");  
    }  
}
```

An interface method declared without a body is implicitly public abstract. Therefore an implementing method cannot reduce the visibility to protected or private.

```
protected void pay() { } // ERROR  
private void pay() { }   // ERROR  
public void pay() { }    // OK
```

17. Multiple Interfaces

```
interface Flyable {  
    void fly();  
}  
  
interface Swimmable {  
    void swim();  
}  
  
class Duck implements Flyable, Swimmable {  
  
    @Override  
    public void fly() {  
        System.out.println("Flying");  
    }  
  
    @Override  
    public void swim() {  
        System.out.println("Swimming");  
    }  
}
```

This is one reason interfaces are powerful: a class can take on multiple contracts/capabilities without inheriting from multiple concrete classes.

18. Abstraction + Polymorphism Together

```
interface Payment {
    void pay();
}

class UPI implements Payment {

    @Override
    public void pay() {
        System.out.println("UPI payment");
    }
}

class CardPayment implements Payment {

    @Override
    public void pay() {
        System.out.println("Card payment");
    }
}

void process(Payment payment) {
    payment.pay();
}

process(new UPI());
process(new CardPayment());
```

The method process depends on the abstraction Payment, not on one concrete payment implementation. Runtime dispatch then selects the correct implementation.

19. Four Pillars in One Example

```
abstract class Animal {

    private String name;

    Animal(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    abstract void sound();
}

class Dog extends Animal {

    Dog(String name) {
        super(name);
    }

    @Override
    void sound() {
        System.out.println("Bark");
    }
}

Animal a = new Dog("Bruno");
a.sound();
```

Encapsulation: private String name protects state and getName() provides controlled access.

Inheritance: Dog extends Animal.

Abstraction: Animal declares abstract behavior sound().

Polymorphism: Animal a = new Dog(...) lets the caller use the parent/abstract type while runtime dispatch selects Dog.sound().

20. Phase 4 Review - Your Earlier Challenge, Correctly Analyzed

```
interface Payment {  
    void pay();  
  
    default void receipt() {  
        System.out.println("Receipt");  
    }  
}  
  
abstract class OnlinePayment {  
    protected double amount;  
  
    OnlinePayment(double amount) {  
        this.amount = amount;  
    }  
  
    abstract void validate();  
}  
  
class UPI extends OnlinePayment implements Payment {  
    UPI(double amount) {  
        super(amount);  
    }  
  
    @Override  
    void validate() {  
        System.out.println("UPI validation");  
    }  
  
    @Override  
    public void pay() {  
        System.out.println("UPI payment");  
    }  
}  
  
OnlinePayment p = new UPI(500);
```

Q1 - Why is `new OnlinePayment(500)` illegal? Because `OnlinePayment` is abstract and cannot be directly instantiated.

Q2 - Why `super(amount)`? It invokes the immediate parent's constructor and initializes the inherited parent portion of the `UPI` object.

Q3 - Does `p.validate()` work? Yes. The reference type `OnlinePayment` declares `validate()`, so the call is allowed; at runtime the actual `UPI` implementation runs.

Q4 - Why does `p.pay()` fail to compile? Because the reference type is `OnlinePayment`, and that type does not declare `pay()`. The fact that the actual object is `UPI` does not expand the compile-time member set of the reference.

Q5 - Why can `p.amount` work? With this exact example, it can be accessed when the calling code is in the same package or when the access occurs through an appropriate subclass context. If `amount` were private, code outside `OnlinePayment` could not access it directly; a getter or protected/public method would be needed.

Q6 - The four pillars: Encapsulation -> protected/internal state plus controlled behavior; Inheritance -> `UPI` extends `OnlinePayment`; Abstraction -> abstract class + interface contracts; Polymorphism -> `OnlinePayment` reference holding a `UPI` object.

21. High-Value Rules to Remember

ABSTRACTION
-> expose what matters

-> hide unnecessary implementation detail

ABSTRACT CLASS

-> cannot be instantiated
-> can have fields, constructors, concrete methods,
abstract methods, static members

INTERFACE

-> contract/type/capability
-> multiple interfaces can be implemented
-> modern interfaces may have abstract, default,
static, and private methods

POLYMORPHISM

-> abstraction works especially well with parent/interface references
-> actual object chooses overridden instance-method behavior

22. Interview-Ready Answer

What is abstraction in Java?

"Abstraction is the process of exposing the essential behavior of an object while hiding unnecessary implementation details. In Java, it is mainly achieved using abstract classes and interfaces. Abstract classes can provide both shared implementation and abstract behavior, while interfaces define contracts that multiple classes can implement."

Don't stop there in an interview. Give a tiny example such as `Payment p = new UPI(); p.pay();` and explain that the caller depends on the `Payment` contract, not the internal `UPI` implementation.

23. Phase 4 Final Challenge

```
interface Notification {  
    void send(String message);  
  
    default void log() {  
        System.out.println("Notification sent");  
    }  
}  
  
abstract class UserNotification {  
    protected String recipient;  
  
    UserNotification(String recipient) {  
        this.recipient = recipient;  
    }  
  
    abstract void validateRecipient();  
}  
  
class EmailNotification  
    extends UserNotification  
    implements Notification {  
  
    EmailNotification(String recipient) {  
        super(recipient);  
    }  
  
    @Override  
    void validateRecipient() {  
        System.out.println("Validating email");  
    }  
  
    @Override  
    public void send(String message) {  
        System.out.println("Sending email: " + message);  
    }  
}
```

```

    }
}

UserNotification n = new EmailNotification("user@example.com");

```

Before moving on, you should be able to answer without running the code:

1. Why can you not instantiate UserNotification?
2. Why is super(recipient) necessary?
3. Does n.validateRecipient() compile? What runs at runtime?
4. Why does n.send("Hi") fail to compile?
5. What would change if the reference were Notification n = new EmailNotification(...)? Which methods become directly accessible?
6. Identify all four OOP pillars in the example.

24. Phase 4 Memory Map

```

ABSTRACTION
|
+-- Abstract class
|   +-- shared state
|   +-- constructors
|   +-- concrete behavior
|   +-- abstract behavior
|
+-- Interface
    +-- contract
    +-- multiple implementation
    +-- default methods
    +-- static methods
    +-- private helpers

ABSTRACTION + POLYMORPHISM
|
+-- Parent/interface reference
+-- Concrete child implementation
+-- Caller depends on the abstraction

```

After this phase, the four pillars should no longer feel like isolated definitions. They form one design system: **encapsulation controls state, inheritance forms a hierarchy, abstraction defines the useful contract, and polymorphism lets different concrete objects fulfill that contract.**